

Лабораторна робота №2

Unit-тестування асинхронного коду з Jest

Мета роботи: набуття практичних навичок написання автоматизованих тестів для асинхронного TypeScript-коду з використанням фреймворку Jest.

Хід роботи:

3.1. Завдання 1: Типізація функцій через generics

Результат виконання:

```
const result : string = await retryOperation(async () : Promise<string> => {
```

```
const processed : number[] = await processInBatches(
```

```
console.log('\n=== Тест 4: Timeout ===');
try {
  await raceWithTimeout(delay(200), 100);
} catch (err) {
```

```
export async function raceWithTimeout<void>(
  promise: Promise<void>,
  timeoutMs: number,
): Promise<void>
```

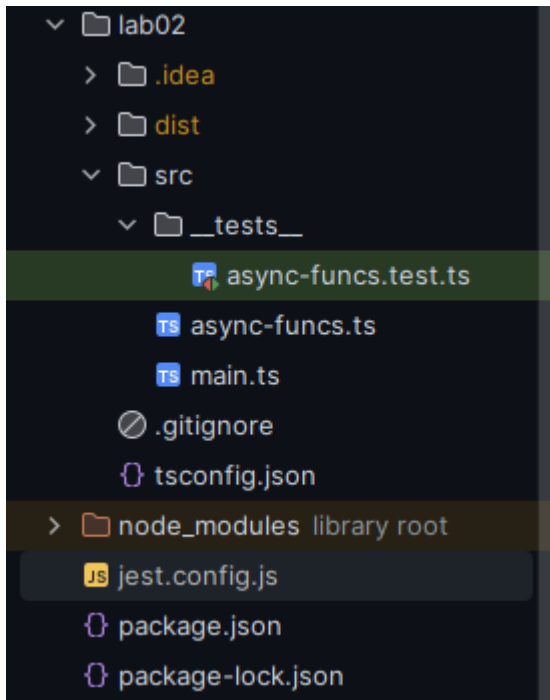
```
у, якщо він заверши
міс не встиг заверш
```

ts lab02/src/async-funcs.ts

3.2. Завдання 2: Налаштування проєкту для тестування

Результат виконання:

					ДУ «Житомирська політехніка».26.121.03.000 – Лр2			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Гуринчук С.М.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Фуріхата Д. В.						Аркушів
Керівник								1
Н. контр.								10
Зав. каф.							ФІКТ Гр. ІПЗ-24-2[1]	



3.3. Завдання 3: Тести для fetchUserProfiles

```
describe('fetchUserProfiles', () => {
  beforeEach(() => {
    jest.useFakeTimers();
  });

  afterEach(() => {
    jest.useRealTimers();
  });

  it('має повертати порожній масив, якщо передано порожній масив ID', async () => {
    const result = await fetchUserProfiles([]);
    expect(result).toEqual([]);
  });

  it('має повертати масив профілів з правильними даними', async () => {
    const promise = fetchUserProfiles(['1', '2']);

    await jest.advanceTimersByTimeAsync(150);

    const result = await promise;

    expect(result).toHaveLength(2);
    expect(result[0]).toEqual({
      id: '1',
      name: 'User 1',
      email: 'user1@example.com'
    });
  });

  it('має успішно виконуватись (resolve) з правильним об'єктом', async () => {
    const promise = fetchUserProfiles(['99']);

    await jest.advanceTimersByTimeAsync(150);
  });
});
```

		Гуринчук С.М.			ДУ «Житомирська політехніка».25.121.01.000 – Лр2	Арк.
		Фуріхата Д. В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    await expect(promise).resolves.toEqual([
      { id: '99', name: 'User 99', email: 'user99@example.com' }
    ]);
  });
});

```

Результат виконання:

PASS lab02/src/__tests__/async-funcs.test.ts

fetchUserProfiles

- ✓ має повертати порожній масив, якщо передано порожній масив ID (7 ms)
- ✓ має повертати масив профілів з правильними даними (12 ms)
- ✓ має успішно виконуватись (resolve) з правильним об'єктом (4 ms)

3.4. Завдання 4: Тести для retryOperation

```

describe('retryOperation', () => {
  let consoleSpy: jest.SpyInstance;

  beforeEach(() => {
    jest.useFakeTimers();

    consoleSpy = jest.spyOn(console, 'log').mockImplementation(() => {});
  });

  afterEach(() => {
    jest.useRealTimers();
    consoleSpy.mockRestore();
  });

  it('має успішно виконатися з першої спроби', async () => {
    const operation = jest.fn().mockResolvedValue('Успіх');

    const result = await retryOperation(operation, 3);

    expect(result).toBe('Успіх');
    expect(operation).toHaveBeenCalledTimes(1);
    expect(consoleSpy).toHaveBeenCalledWith('Спроба 1...');
  });

  it('має повторити спробу після помилки та успішно виконатись', async () => {
    const operation = jest.fn()
      .mockRejectedValueOnce(new Error('Помилка 1'))
      .mockResolvedValueOnce('Успіх з 2 спроби');

    const promise = retryOperation(operation, 3);

    await jest.advanceTimersByTimeAsync(1000);

    const result = await promise;

    expect(result).toBe('Успіх з 2 спроби');
    expect(operation).toHaveBeenCalledTimes(2);
  });
});

```

		Гуринчук С.М.			ДУ «Житомирська політехніка».25.121.01.000 – Пр2	Арк.
		Фуріхата Д. В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    expect(consoleSpy).toHaveBeenCalledWith('Помилка, чекаємо 1000мс...');
  });

  it('має викинути помилку після вичерпання всіх спроб (дефолтний параметр)',
  async () => {
    const error = new Error('Фатальна помилка');

    const operation = jest.fn().mockRejectedValue(error);

    const promise = retryOperation(operation);

    const assertion = expect(promise).rejects.toThrow('Фатальна помилка');

    await jest.advanceTimersByTimeAsync(2000);

    await assertion;
    expect(operation).toHaveBeenCalledTimes(3);
  });
});

```

Результат виконання:

```

retryOperation
  ✓ має успішно виконатися з першої спроби (6 ms)
  ✓ має повторити спробу після помилки та успішно виконатись (4 ms)
  ✓ має викинути помилку після вичерпання всіх спроб (дефолтний параметр) (24 ms)
  ✓ має викинути Unknown error, якщо передати maxRetries = 0 (13 ms)

```

3.5. Завдання 5: Тести для processInBatches

```

describe('processInBatches', () => {
  let consoleSpy: jest.SpyInstance;

  beforeEach(() => {
    consoleSpy = jest.spyOn(console, 'log').mockImplementation(() => {});
  });

  afterEach(() => {
    consoleSpy.mockRestore();
  });

  it('має коректно обробляти дані партіями та повертати об'єднаний результат',
  async () => {
    const processor = jest.fn(async (batch: number[]) => batch.map(n => n *
2));

    const result = await processInBatches([1, 2, 3, 4, 5], 2, processor);

    expect(result).toEqual([2, 4, 6, 8, 10]);

    expect(processor).toHaveBeenCalledTimes(3);
  });
});

```

		Гуринчук С.М.			ДУ «Житомирська політехніка».25.121.01.000 – Пр2	Арк.
		Фуріхата Д. В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    it('має повертати порожній масив, якщо на вхід передано порожній масив', async
    () => {
        const processor = jest.fn();

        const result = await processInBatches([], 2, processor);

        expect(result).toEqual([]);
        expect(processor).not.toHaveBeenCalled();
    });

    it('має логувати прогрес обробки кожної партії', async () => {
        const processor = jest.fn(async (batch: number[]) => batch);

        await processInBatches([1, 2, 3], 2, processor);

        expect(consoleSpy).toHaveBeenCalledWith(expect.stringContaining('Обробка
партії 1/2'));
        expect(consoleSpy).toHaveBeenCalledWith(expect.stringContaining('Обробка
партії 2/2'));
    });
});

```

Результат виконання:

processInBatches

- ✓ має коректно обробляти дані партіями та повертати об'єднаний результат (2 ms)
- ✓ має повертати порожній масив, якщо на вхід передано порожній масив (2 ms)
- ✓ має логувати прогрес обробки кожної партії (3 ms)

3.6. Завдання 6: Тести для raceWithTimeout

```

describe('raceWithTimeout', () => {
    beforeEach(() => {
        jest.useFakeTimers();
    });

    afterEach(() => {
        jest.useRealTimers();
    });

    it('має повертати результат промісу, якщо він завершився швидше за таймаут',
    async () => {
        const fastPromise = Promise.resolve('Успіх');

        const result = await raceWithTimeout(fastPromise, 1000);

        expect(result).toBe('Успіх');
    });

    it('має викидати помилку, якщо проміс не встиг завершитися за відведений час',
    async () => {
        const slowPromise = new Promise(resolve => setTimeout(resolve, 2000));

        const racePromise = raceWithTimeout(slowPromise, 1000);
    });
});

```

		Гуринчук С.М.			ДУ «Житомирська політехніка».25.121.01.000 – Пр2	Арк.
		Фуріхата Д. В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    const assertion = expect(racePromise).rejects.toThrow('Operation timed out
after 1000ms');

    await jest.advanceTimersByTimeAsync(1000);

    await assertion;
  });
});

```

Результат виконання:

raceWithTimeout

- ✓ має повертати результат промісу, якщо він завершився швидше за таймаут (2 ms)
- ✓ має викидати помилку, якщо проміс не встиг завершитися за відведений час (6 ms)

Результат coverage:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
async-funcs.ts	100	100	100	100	
Test Suites: 1 passed, 1 total					
Tests: 12 passed, 12 total					
Snapshots: 0 total					

Висновок: набув практичних навичок написання автоматизованих тестів для асинхронного TypeScript-коду з використанням фреймворку Jest.

Git: <https://git.ztu.edu.ua/ipz/2024-2028/ipz-24-2/Hurynchuk-Sviatoslav/nodejs-labs>

		Гуринчук С.М.			ДУ «Житомирська політехніка».25.121.01.000 – Лр2	Арк.
		Фуріхата Д. В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		